Министерство науки и высшего образования Пензенский государственный университет Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5

по курсу «Логика и основы алгоритмизации в инженерных задачах» на тему «Определение характеристик графов»

Выполнили: ст. гр. 25ВВВ2

Потёмкин Д.С.

Гладков А.М.

Проверил: к.т.н., доцент

Юрова О.В.

к.т.н.

Деев М.В.

Пенза 2026 г.

Название: Определение характеристик графов

Цель работы: Изучение способов представления графов в памяти ЭВМ (матрица смежности и матрица инцидентности). Получение навыков генерации случайных графов с заданной вероятностью появления ребра (включая петли) и определения основных характеристик графа: размера, степеней вершин, а также поиска изолированных, концевых и доминирующих вершин.

Лабораторное задание

1. Сгенерировать (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Вывести матрицу на экран. Учесть вероятность появления ребра (задаётся пользователем) и возможность существования петель.
2. Определить размер графа G, используя матрицу смежности графа.
3. Найти изолированные, концевые и доминирующие вершины.
4. *(Задание 2*)* Построить для графа Gматрицу инцидентности.
5. Определить размер графа G, используя матрицу инцидентности графа.
6. Найти изолированные, концевые и доминирующие вершины по матрице инцидентности.

Пояснительный текст к программам

Все программы разработаны на языке программирования **Си** с использованием стандартных библиотек. Для ввода-вывода используется библиотека stdio.h, для работы с динамической памятью — stdlib.h, для генерации случайных чисел — time.h и stdlib.h.

Граф Gсодержит непустое множество nвершин Vи множество рёбер E. Размер графа G— это мощность множества рёбер $|E(G)|$, то есть количество рёбер графа. Степенью вершины называется число инцидентных ей рёбер. Вершина со степенью 0 называется изолированной, со степенью 1 — концевой. Вершина, смежная с каждой другой вершиной графа, называется доминирующей.

Матрица смежности — квадратная матрица Aразмера $n \times n$, где $A[i][j] = 1$, если между вершинами iи jесть ребро, и 0в противном случае. Для неориентированного графа матрица симметрична. Петля (ребро из вершины в саму себя) записывается как $A[i][i] = 1$.

Матрица инцидентности — матрица размера $n \times |E|$, где строки соответствуют вершинам, а столбцы — рёбрам. Для неориентированного графа в

столбце ребра ставятся две единицы — в строках его концов. Петля обозначается двойкой в строке соответствующей вершины.

Переменные:

- n — количество вершин графа;
- p — вероятность появления ребра (от 0.0 до 1.0);
- adj — матрица смежности (динамическая, int**);
- inc — матрица инцидентности (динамическая, int**);
- size — размер графа (количество рёбер);
- edges — количество рёбер, найденное при построении матрицы инцидентности;
- choice — переменная для выбора пункта меню.

Для графа реализованы функции создания матрицы, генерации матрицы смежности, вывода матрицы, определения размера графа, анализа вершин, построения матрицы инцидентности и освобождения памяти:

- функция createMatrix(n, m) — выделение памяти под матрицу $n \times m$;
- функция freeMatrix(mat, n) — освобождение памяти матрицы;
- функция generateAdjMatrix(n, p, adj) — генерация матрицы смежности;
- функция printMatrix(mat, n, m, title) — вывод матрицы;
- функция getSizeFromAdj(n, adj) — размер графа по матрице смежности;
- функция analyzeByAdj(n, adj) — анализ вершин по матрице смежности;
- функция buildIncMatrix(n, adj, inc) — построение матрицы инцидентности;
- функция analyzeByInc(n, edges, inc) — анализ вершин по матрице инцидентности;
- функция inputGraphParams(&n, &p) — ввод и проверка параметров графа.

Пояснения к Заданию 1 (матрица смежности)

Основные операции

Функция выделения памяти под матрицу createMatrix(n, m):

```
функция createMatrix(n, m):
    mat = выделить память(n * sizeof(int*))
    если mat = NULL:
        вывести "Ошибка выделения памяти!"
        выход(1)
    для i от 0 до n-1:
        mat[i] = выделить память с обнулением(m * sizeof(int))
        если mat[i] = NULL:
            вывести "Ошибка выделения памяти!"
```

```
            выход(1)
    вернуть mat
конец функции
```

Функция генерации матрицы смежности generateAdjMatrix(n, p, adj):

```
функция generateAdjMatrix(n, p, adj):
    для i от 0 до n-1:
        для j от 0 до n-1:
            если i = j:
                // Петля
                если случайное число от 0 до 1 < p:
                    adj[i][j] = 1
                иначе:
                    adj[i][j] = 0
            иначе если i < j:
                // Обычное ребро
                если случайное число от 0 до 1 < p:
                    adj[i][j] = 1
                    adj[j][i] = 1
                иначе:
                    adj[i][j] = 0
                    adj[j][i] = 0
конец функции
```

Функция определения размера графа getSizeFromAdj(n, adj):

```
функция getSizeFromAdj(n, adj):
    size = 0
    для i от 0 до n-1:
        если adj[i][i] = 1:
            size = size + 1        // Петля
        для j от i+1 до n-1:
            если adj[i][j] = 1:
                size = size + 1    // Обычное ребро
    вернуть size
конец функции
```

Функция анализа вершин analyzeByAdj(n, adj):

```
функция analyzeByAdj(n, adj):
    для i от 0 до n-1:
        step = 0
        neighbors = 0
        для j от 0 до n-1:
            если adj[i][j] = 1:
                step = step + 1
                если i != j:
                    neighbors = neighbors + 1
        если adj[i][i] = 1:
            step = step + 1        // Петля даёт +1 (итого 2)
        вывести "Вершина", i+1, "степень", step
        если step = 0:
            вывести "(изолированная)"
        если step = 1:
            вывести "(концевая)"
```

```
        если neighbors = n-1:
            вывести "(доминирующая)"
конец функции
```

Главная функция main():

```
функция main():
    srand(текущее время)
    пока истина:
        вывести меню:
            1 - Задание 1 (матрица смежности)
            2 - Задание 2* (матрица инцидентности)
            0 - выход
        ввести choice

        если choice = 0:
            прервать цикл

        если choice = 1 или choice = 2:
            освободить старую матрицу adj
            если не inputGraphParams(n, p):
                продолжить
            adj = createMatrix(n, n)
            generateAdjMatrix(n, p, adj)

        если choice = 1:
            вывести "Задание 1"
            printMatrix(adj, n, n, "Матрица смежности")
            вывести "Размер графа:", getSizeFromAdj(n, adj)
            analyzeByAdj(n, adj)
        иначе если choice = 2:
            вывести "Задание 2*"
            max_edges = n * (n + 1) / 2
            inc = createMatrix(n, max_edges)
            edges = buildIncMatrix(n, adj, inc)
            printMatrix(inc, n, edges, "Матрица инцидентности")
            вывести "Размер графа:", edges
            analyzeByInc(n, edges, inc)
            freeMatrix(inc, n)
        иначе:
            вывести "Неверный пункт меню"

    freeMatrix(adj, n)
конец функции
```

Пояснения к Заданию 2 (матрица инцидентности)

Функция построения матрицы инцидентности buildIncMatrix(n, adj, inc):

```
функция buildIncMatrix(n, adj, inc):
    edges = 0
    для i от 0 до n-1:
        для j от 0 до n-1:
            если adj[i][j] = 1 и i = j:
                inc[i][edges] = 2      // Петля
```

```
                edges = edges + 1
            иначе если adj[i][j] = 1 и i < j:
                inc[i][edges] = 1
                inc[j][edges] = 1
                edges = edges + 1
    вернуть edges
конец функции
```

Функция анализа вершин по матрице инцидентности analyzeByInc(n, edges, inc):

```
функция analyzeByInc(n, edges, inc):
    для i от 0 до n-1:
        step = 0
        neighbors[20] = {0}
        countNeighbors = 0
        для j от 0 до edges-1:
            если inc[i][j] = 1:
                step = step + 1
                для k от 0 до n-1:
                    если k != i и inc[k][j] = 1 и neighbors[k] = 0:
                        neighbors[k] = 1
                        countNeighbors = countNeighbors + 1
            иначе если inc[i][j] = 2:
                step = step + 2        // Петля
        вывести "Вершина", i+1, "степень", step
        если step = 0:
            вывести "(изолированная)"
        если step = 1:
            вывести "(концевая)"
        если countNeighbors = n-1:
            вывести "(доминирующая)"
конец функции
```

Результаты работы программ

Результаты задания 1 показаны на рисунке 1

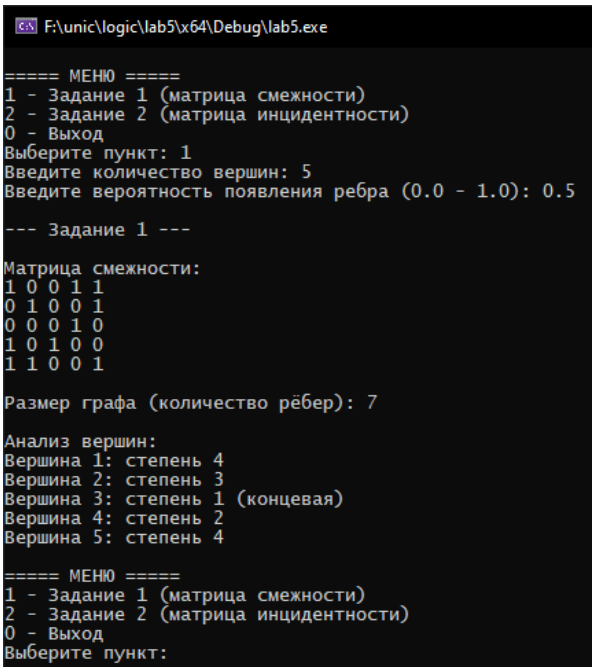


Рисунок 1

Результаты задания 2 показаны на рисунке 2

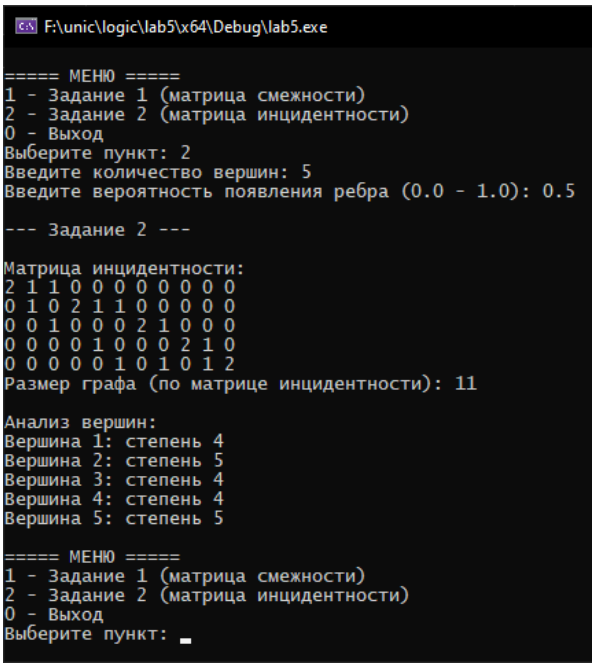


Рисунок 2

Выводы

В ходе выполнения лабораторной работы были разработаны программы на языке программирования Си, демонстрирующие работу с графами и их характеристиками на основе матрицы смежности и матрицы инцидентности.

Задание 1: Успешно реализована генерация матрицы смежности для неориентированного графа с заданной вероятностью появления ребра. Программа корректно учитывает петли (вершина соединена сама с собой), заполняя главную диагональ матрицы. Размер графа определяется подсчётом рёбер выше главной диагонали и петель на ней, что исключает двойной учёт симметричных элементов. Реализован анализ вершин: найдены степени, изолированные, концевые и доминирующие вершины. Степень вершины с петлёй корректно увеличивается на 2.

Задание 2*: На основе матрицы смежности построена матрица инцидентности. Для обычных рёбер в соответствующих строках ставятся единицы, для петель — двойка в строке вершины. Размер графа определяется по количеству столбцов матрицы инцидентности и совпадает с результатом Задания 1. Анализ вершин по матрице инцидентности также позволяет найти изолированные, концевые и доминирующие вершины: степень вершины равна сумме элементов её строки, а доминирующая вершина имеет $n - 1$различных соседей.

Все программы работают без ошибок, результаты выполнения совпадают с ожидаемыми. Продемонстрирована корректная работа с динамической памятью: выделение через malloc() и calloc() (с обнулением) и освобождение через free() во всех операциях, включая повторный запуск заданий и выход из программы. Использование функций createMatrix() и freeMatrix() позволило избежать дублирования кода и утечек памяти.